

NATIVE + CLOUDFLARE WORKERS

Akamata

Portable Zig

Backend Framework

Zig functions. Typed APIs. Explicit ownership.

The application is yours

Write Zig. Build a backend.

Choose Native or Cloudflare Workers.

Application

Native

VPS / local / Container

Cloudflare Workers

WASM / Edge

An ordinary function becomes an API

```
fn hello() []const u8 {  
    return "Hello, Akamata!";  
}
```

```
pub const Application = ak.App(.{  
    .routes = .{  
        ak.get("/", hello),  
    },  
});
```

Application declaration from tests/docs/minimal.zig. Allocator and entry point omitted.

Zig types define the HTTP contract

Zig function + struct + error set

Explicit inputs

Path / Query / Header / Cookie
JSON DTO / Validation / Principal

Typed results

Response / HTTP error mapping
Security / Capability metadata

Router / OpenAPI / TypeScript client / Tests

Handlers and explicit endpoint metadata share the existing HTTP core.

Room to grow

01 Function

03 Database

05 Storage / Queue

02 Typed JSON API

04 Authentication

06 Realtime

Start without Context. Use explicit Context, middleware and provider owners when you need control.

One application, explicit providers

Application / Portable Contract

Native

SQLite / Turso

Filesystem

jobs.Queue

Native realtime Service

Workers

D1 / Turso

R2

Queues

Durable Objects gateway

Container uses Native providers. Entry points, owners and platform limits remain explicit.

Native Threaded performance

1 65k

/hello

HTTP 200 req/s

1 64k

/echo

HTTP 200 req/s

87k

/db/1

HTTP 200 req/s

A local runtime snapshot. No ranking or cross-framework comparison.

macOS 27 / Apple M2 / Zig 0.17.0 / ReleaseFast / Threaded

32 keep-alive / loopback / oha 1.15.0 / 3 × 3s per workload / mean of all rounds

[Raw data, commit/binary SHA and reproduction](#)

The same domain code at the edge

Workers WASM + managed JavaScript glue

— — — — —
D1 R2 Queues Durable Objects

Share DTOs, error meanings, event protocols and portable service effects.
Configure bindings and owners for the target.

Offline adapter/host contracts are tested. Live Cloudflare certification is not claimed.

Test the application in process

```
var client = app.client(allocator);  
var response = try client.post("/users")  
    .json(.{ .name = "Alice" }).send();  
defer response.deinit();  
try response.expectStatus(.created);
```

MemoryStore / QueueRecorder / Principal injection

The shared DX fixture runs on Native and Workers WASM. Test doubles do not prove live delivery.

The development workflow

init

Minimal app

dev

Build / run / watch

test

Application tests

check

Project validation

deploy

Explicit target

`akamata init myapp`

Inspect routes, OpenAPI and capabilities as the application grows.

Commands require project configuration. Deployment does not implicitly create resources.

Portability reaches deployment

Application Capability Provider Binding Owner Deployment

Declare what the app needs.

Validate how the target provides it.

Local validation, remote reachability and application readiness are different evidence levels.

What runs where

	Native	Workers	Container
HTTP	Threaded	WASM / fetch	Threaded
DB	SQLite / Turso	D1 / Turso	SQLite / Turso
Storage	Filesystem	R2	Filesystem *
Queue	jobs.Queue	Queues	jobs.Queue
Realtime	Native Service	DO gateway	Native Service

* Persistent DB/filesystem need mounted volumes. Streaming, retry and gateway details differ by platform.

Your first application

```
akamata init myapp  
cd myapp  
akamata dev
```

[Getting Started](#) [Guides / Examples](#) [GitHub](#)

Install the CLI first. For this main-branch API, follow the local dependency override in Getting Started.